

Medium-Scale PeTTa Memory for ProtomegaTron: Detailed Design and Step-by-Step Implementation Plan

OmegaClaw / ZeroBot working notes for Ben Goertzel

2026-06-27

Abstract

This ASCII-only LaTeX note specifies a conservative design and implementation plan for adding an intermediate-scale PeTTa memory layer to ProtomegaTron. The layer sits between volatile working memory and broad ChromaDB long-term memory. Its purpose is to preserve structured, queryable, provenance-bearing cognitive state: episodes, entities, commitments, hypotheses, open questions, action boundaries, observations, and distillation links. The plan is intentionally incremental: first add a file-backed atom store and read-only query surface, then add selective extraction, then add distillation and eventual integration into the OmegaClaw loop. The 2026-06-27 revision makes the design explicitly PLN-ready: the same medium memory should later support inference over a normalized, truth-valued AtomSpace view without confusing quoted conversation, observed events, decisions, hypotheses, and derived beliefs.

1 Motivation

1.1 Revision summary

This version folds in three constraints that should shape the implementation:

1. The canonical unit is a **MemoryCluster**, not an isolated atom line.
2. Observed events, quoted utterances, commitments, hypotheses, decisions, and derived beliefs must remain epistemically distinct.
3. The store is append-only; current state is a derived view over status, supersession, and truth-value events.

The current OmegaClaw memory architecture, as inspected in the local ProtomegaTron/OmegaClaw checkout, has four important memory surfaces:

1. Working memory, represented by **pin**, useful for immediate task state.
2. The episodic text trace, **memory/history.metta**, whose tail is injected into prompts and whose neighborhoods can be retrieved by **episodes**.
3. ChromaDB long-term memory, accessed by **remember** and **query**, useful for semantic recall across longer time spans.
4. Per-invocation MeTTa/AtomSpace reasoning, useful but not persistent across separate calls unless the agent manually carries atoms forward.

This leaves a gap. ProtomegaTron needs a memory layer that is more precise than embedding recall, more persistent than a single pin or one fresh AtomSpace call, and more structured than a raw text history tail. The proposed layer is a medium-scale PeTTa memory, abbreviated here as MM.

2 Design goal

Definition 1 (Medium-scale PeTTa memory). *A medium-scale PeTTa memory is a bounded, persistent, typed atom store*

$$MM_t = \{a_1, a_2, \dots, a_n\}$$

where each atom or small atom cluster records a currently useful cognitive item: an episode, entity, commitment, hypothesis, question, boundary, observation, artifact, or provenance relation. It is intended for active structured cognition, not for permanent archival storage of everything the agent sees.

Design Principle 1 (Selective atomization). *Only high-value events should enter MM. The store should contain facts that are likely to affect near-future reasoning, task continuity, self-scrutiny, or Hyperseed formalization. It should not become a complete transcript mirror.*

Design Principle 2 (Provenance first). *Every nontrivial assertion should either include direct provenance or link to a provenance atom. A false or stale symbolic assertion without provenance is more dangerous than an unstructured text memory, because it looks more formal than it is.*

Design Principle 3 (Epistemic role separation). *Atoms that record what happened should be separated from atoms that claim what is true. A message from Ben, a quote from an LLM, a derived belief, a standing decision, and a speculative hypothesis should have different predicates and should not be collapsed into a single untyped fact. This is especially important once PLN inference is run over the memory AtomSpace: quoted claims may provide evidence, but they should not automatically become premises.*

Design Principle 4 (Append-only audit semantics). *The canonical store should append new status, salience, and truth-value events rather than silently mutating old atoms. Current status can be derived from the latest non-superseded status event. This preserves an audit trail and makes contradictions inspectable instead of hidden.*

Design Principle 5 (Bounded autonomy). *The first implementation should not grant ProtomegaTron new external authority. It should only write a local structured memory file and read/query that file. Telegram, GitHub, shell, paid compute, and filesystem permissions remain governed by existing boundaries.*

Design Principle 6 (PLN-ready AtomSpace). *Ben noted on 2026-06-27 that later versions should run PLN inference on this memory AtomSpace. Therefore version 1 should avoid ad hoc string-only structures that would be hard to promote into PLN premises. Atoms should use stable predicates, explicit truth values for interpretive claims when appropriate, provenance links, and status events that can later be loaded into a reasoning context without lossy translation.*

3 Placement in the architecture

The intended memory stack is:

$$WM \rightarrow MM \rightarrow LTM$$

where:

- WM is working memory: pins, current loop state, current prompt tail.
- MM is medium PeTTa memory: structured atoms over recent and active concerns.
- LTM is ChromaDB plus durable project records and curated Markdown.

Operationally, a turn may use the layers as follows:

1. Receive message or autonomous wake event.
2. Retrieve relevant MM atoms by type, entity, status, or recency.
3. Retrieve relevant ChromaDB memories if broad semantic context is needed.
4. Construct prompt context from mandate, skills, selected history, selected MM atoms, and recent results.
5. Let the LLM propose skill calls and optional memory updates.
6. Execute safe skill calls.
7. Append high-value structured atoms to MM.
8. Periodically distill older MM atoms into ChromaDB or project notes.

3.1 Canonical log and derived views

Version 1 should distinguish four products:

1. **MM-log**: append-only canonical atom journal. This is the source of truth and should be small enough to inspect with ordinary tools.
2. **MM-index**: rebuildable query acceleration, such as type, id, about, status, and time indexes. It can be deleted and regenerated.
3. **MM-prompt-view**: bounded, human-readable context injected into LLM prompts. It should never be treated as canonical.
4. **MM-pln-view**: normalized, typed, truth-valued atoms safe to load as PLN premises or evidence. It should exclude raw quoted claims unless a rule explicitly promotes them into evidence-bearing beliefs.

This separation lets the same substrate support auditability, ordinary prompt continuity, and later probabilistic reasoning without requiring all consumers to share the same representation.

4 Minimal atom schema

The schema should be small enough to remember and inspect by eye. The first version should use flat, readable predicates rather than deep ontological machinery.

4.1 Identifiers

Use stable, opaque identifiers:

```
(Episode e-20260627-121018-001)
(Entity ent-protomegatron)
(Question q-medium-memory-decay)
(Hypothesis h-medium-memory-useful)
(Commitment c-write-medium-memory-plan)
(Artifact art-medium-memory-plan-tex)
(Boundary b-telegram-allowlist)
(MemoryCluster mc-20260627-121018-001)
(Source src-loop-metta-16d380d)
```

Identifier construction rule:

```
<kind>-<yyyymmdd>-<hhmmss>-<counter>
```

for event-like records, and stable slugs for durable entities or code anchors.

4.2 Cluster envelope

The canonical write unit should be a small cluster, not an isolated line. This makes append-only updates, provenance inspection, and later PLN loading safer.

```
(MemoryCluster mc1)
(ClusterType mc1 episode-record)
(ClusterOpenedAt mc1 "2026-06-27 12:10 PDT")
(ClusterSource mc1 src-telegram-history-20260627)
(Contains mc1 e1)
(Contains mc1 c1)
(Contains mc1 q1)
(ClusterStatus mc1 active)
```

Cluster conventions:

1. Every append writes one complete cluster.
2. A later correction writes a new cluster and links it with **Supersedes**, rather than editing the old cluster in place.
3. Queries may return whole clusters even when only one contained atom matched.
4. The PLN view may select only premise-safe atoms from a cluster.

4.3 Epistemic roles

The first schema should reserve explicit predicates for epistemic roles, and should tag clusters or atoms when the role is not obvious from the predicate name. This makes the memory useful for ordinary retrieval now and safer for PLN inference later.

```
(ObservedEvent e1)
(SpeechEvent se1)
(QuotedClaim qc1)
(DerivedBelief b1)
(Decision d1)
(Hypothesis h1)
(OpenQuestion q1)
(Commitment c1)
```

The intended discipline is:

- **ObservedEvent**: something the local system directly observed, such as a Telegram message, file creation, test result, or tool output.
- **SpeechEvent**: a communicative event, usually linking actor, channel, and message id.
- **QuotedClaim**: what an actor or model asserted. It is evidence, not automatically a fact.
- **DerivedBelief**: a normalized proposition that the agent is willing to use as a belief, with a truth value and evidence links.
- **Decision**: an adopted course of action with rationale and provenance.
- **Hypothesis**: a plausible but unsettled proposition, normally with a lower confidence than a decision or directly observed fact.

4.4 Episode atoms

```
(Episode e1)
(AtTime e1 "2026-06-27 12:10 PDT")
(Channel e1 telegram-group)
(Actor e1 Ben)
(ObservedBy e1 ProtomegaTron)
(Said e1 Ben "Great please progress w this...")
(About e1 ProtomegaTronDesign)
(About e1 MediumPeTTaMemory)
(HasStatus e1 recorded)
(HasProvenance e1 src-telegram-history-20260627)
(EpistemicRole e1 observed-event)
```

For PLN-readiness, prefer the more explicit event-plus-claim form when the content of a message may later be interpreted as evidence:

```
(ObservedEvent e1)
(SpeechEvent se1)
(PartOf se1 e1)
(AtTime se1 "2026-06-27 14:02 PDT")
(Channel se1 telegram-group)
(Speaker se1 Ben)
(MessageId se1 telegram-msg-505)
(RawUtterance se1 "Note that later we will be running PLN inference...")
```

```

(QuotedClaim qc1)
(ClaimSource qc1 se1)
(ClaimText qc1 "Later versions should run PLN inference on this memory AtomSpace")
(About qc1 MediumPeTTaMemory)
(About qc1 PLN)
(ClaimStatus qc1 quoted-unverified)

```

4.5 Commitment atoms

```

(Commitment c1)
(CommittedBy c1 ProtomegaTron)
(CommittedTo c1 Ben)
(CommitmentText c1 "Write an ASCII LaTeX design and implementation plan")
(CommitmentStatus c1 active)
(OpenedBy c1 e1)
(HasDueMode c1 immediate-next-step)
(HasProvenance c1 e1)
(EpistemicRole c1 commitment)

```

When status changes, append status events instead of overwriting the old status:

```

(StatusEvent st1)
(StatusSubject st1 c1)
(StatusValue st1 active)
(StatusTime st1 "2026-06-27 13:51 PDT")
(StatusProvenance st1 e1)

(StatusEvent st2)
(StatusSubject st2 c1)
(StatusValue st2 resolved)
(StatusTime st2 "2026-06-27 14:10 PDT")
(Supersedes st2 st1)
(StatusProvenance st2 art-medium-memory-plan-tex)

```

4.6 Question atoms

```

(OpenQuestion q1)
(QuestionText q1 "How should medium PeTTa memory decay and distill?")
(QuestionStatus q1 active)
(QuestionScope q1 architecture)
(About q1 MediumPeTTaMemory)
(HasProvenance q1 e1)
(EpistemicRole q1 open-question)

```

4.7 Hypothesis atoms

Truth values can be represented as ordinary MeTTa structures initially, without requiring every query to invoke PLN.

```

(Hypothesis h1)
(HypothesisText h1 "Selective medium PeTTa memory improves continuity")
(About h1 ProtomegaTronMemory)
(Confidence h1 (stv 0.80 0.60))
(EvidenceFor h1 e1)
(HypothesisStatus h1 active)
(EpistemicRole h1 hypothesis)

```

For PLN, the proposition should be explicit rather than trapped inside a string:

```

(Hypothesis h1)
(HypothesisContent h1 (Improves SelectiveMediumPeTTaMemory AgentContinuity))
(TruthValue h1 (stv 0.80 0.60))
(EvidenceFor h1 qc1)
(HasProvenance h1 se1)
(StatusSubject st-h1-1 h1)
(StatusValue st-h1-1 active)

```

The string form may still be kept for prompt readability, but the symbolic content is what should be loaded into the PLN view.

4.8 Belief atoms for PLN

Derived beliefs are the main bridge between audit memory and inference memory:

```

(DerivedBelief b1)
(BeliefContent b1 (Requires MediumPeTTaMemory ExplicitTruthValues))
(TruthValue b1 (stv 0.90 0.70))
(EvidenceFor b1 qc1)
(EvidenceFor b1 design-review-20260627)
(BeliefStatus b1 active)
(HasProvenance b1 se1)

```

Rules may later promote quoted claims into evidence-bearing beliefs when a source is trusted in a relevant context, for example:

```

(TrustedSource Ben medium-memory-design)
(CanPromoteToEvidence qc1 b1)

```

but promotion should be explicit and logged, not implicit.

4.9 Salience and retention atoms

Bounded memory requires explicit attention and retention metadata:

```

(SalienceEvent sal1)
(SalienceSubject sal1 c1)
(SalienceValue sal1 0.82)
(SalienceReason sal1 active-task)
(RetentionClass c1 active-task)
(ReviewAfter c1 "2026-06-28")

```

The prompt view should prefer high-salience active items. The distillation pass should prefer low-salience resolved items and stale open questions.

4.10 Boundary atoms

```
(Boundary b1)
(BoundaryName b1 "Telegram allowlist")
(Constrains b1 send-action)
(Constrains b1 receive-action)
(ImplementedIn b1 "channels/telegram.py")
(HasStatus b1 active)
(HasProvenance b1 src-telegram-py-16d380d-local)
(EpistemicRole b1 action-boundary)
```

4.11 Artifact atoms

```
(Artifact art1)
(ArtifactPath art1 "papers/0003-medium-petta-memory-plan/medium_petta_memory_plan.tex")
(ArtifactType art1 latex-source)
(About art1 MediumPeTTaMemory)
(CreatedBy art1 ProtomegaTron)
(HasProvenance art1 c1)
(EpistemicRole art1 artifact-record)
```

4.12 Append-only update atoms

Status changes, confidence revisions, and corrections should be new atoms or new clusters. The current view is computed from the latest non-superseded update.

```
(StatusEvent se1)
(StatusSubject se1 c1)
(StatusValue se1 resolved)
(AtTime se1 "2026-06-27 12:24 PDT")
(HasProvenance se1 art1)

(TruthValueEvent tv1)
(TruthSubject tv1 h1)
(TruthValue tv1 (stv 0.85 0.65))
(Supersedes tv1 tv0)
(HasProvenance tv1 e2)
```

This pattern is more verbose than mutation, but it preserves the audit trail needed for both scientific provenance and later PLN belief revision.

5 File layout

The first prototype should avoid database complexity. Use append-only files under the OmegaClaw project or a local experimental branch:

```

OmegaClaw-Core/
memory/
  medium_memory.metta      # append-only canonical atom log
  medium_memory_index.metta # optional derived index, rebuildable
src/
  medium_memory.metta      # MeTTa API wrapper
  medium_memory.py         # Python helper for simple file operations

```

If implemented first in the research workspace rather than upstream OmegaClaw Core, use:

```

projects/omegaclaw/experiments/medium-memory-prototype/
  medium_memory.metta
  medium_memory.py
  tests/

```

6 API design

The first API should be deliberately small.

6.1 Write API

```

(mm-append $atom)
(mm-append-cluster $cluster)
(mm-record-episode $episode-id $cluster)
(mm-append-status $subject-id $status $provenance)

```

Expected behavior:

1. Validate that the atom is parseable as MeTTa syntax.
2. Reject atoms above a configured size limit.
3. Append with a timestamp comment or explicit `AtTime` atom.
4. Return a success/failure status that can be fed back into the loop.

6.2 Read API

```

(mm-tail $max-chars)
(mm-query-cluster $cluster-id)
(mm-query-type $type $limit)
(mm-query-about $entity $limit)
(mm-query-status $status $limit)
(mm-query-role $epistemic-role $limit)
(mm-query-id $id)

```

The first implementation can use simple text scanning. Later versions can load the file into an AtomSpace or maintain derived indexes.

6.3 View API

The store should expose separate views for separate consumers:

```
(mm-prompt-view $topic $max-chars)
(mm-pln-view $topic $max-atoms)
(mm-audit-view $id $max-chars)
```

Expected behavior:

- `mm-prompt-view` returns a compact textual summary of active commitments, questions, salient observations, and relevant boundaries.
- `mm-pln-view` returns normalized typed atoms with explicit truth values and evidence links. It should not include raw quoted claims as factual premises unless they have been explicitly promoted.
- `mm-audit-view` returns the provenance chain and append-only event history for one id.

This division is a safety feature, not merely an implementation detail: prompt selection, human audit, and PLN inference have different failure modes.

6.4 Distillation API

```
(mm-candidates-for-distill $older-than $limit)
(mm-mark-distilled $id $destination)
(mm-export-summary $ids)
```

Distillation should not delete records initially. It should mark them as distilled and link to the destination: ChromaDB memory id, project Markdown file, Git commit, or formalization note.

7 Prompt integration

The prompt should not include the full MM store. It should include a bounded selection:

1. Active commitments involving Ben or the current project.
2. Active open questions about the current project or agent design.
3. Recent episodes relevant to the current message.
4. Boundaries relevant to proposed actions.
5. Recent failed or surprising observations.

A possible prompt block:

MEDIUM_MEMORY_ACTIVE:

- Commitment c1: write ASCII LaTeX design plan. Status active.
- Question q1: how should medium PeTTa memory decay and distill? Status active.
- Boundary b1: Telegram allowlist constrains send/receive actions. Status active.

This human-readable summary can be generated from atoms. The atoms remain the source of truth; the summary is a prompt-facing view.

8 Step-by-step implementation plan

Milestone 1 (Step 0: freeze scope). *Goal: implement a local experimental medium memory that cannot affect external actions except by improving prompt context.*

Deliverables:

- A short design note, this file.
- A minimal schema file with example atoms.
- A decision on prototype location: local experiment first, then OmegaClaw Core.

Exit criteria:

- No new credentials, services, or external write authority are introduced.
- The atom schema fits on one page and has examples.

Milestone 2 (Step 1: create append-only store). *Implement `memory/medium_memory.metta` and helper functions for appending validated clusters and atoms.*

Tasks:

1. Add `src/medium_memory.py` with `append_atom`, `append_cluster`, `tail`, and `parse-check` helpers.
2. Add `src/medium_memory.metta` wrappers around those Python calls.
3. Add limits: maximum atom length, maximum cluster length, maximum tail chars.
4. Add cluster-envelope checks: every cluster needs a `MemoryCluster` id, `ClusterType`, timestamp, provenance/source, and at least one `Contains` link.
5. Add tests for successful cluster append, malformed atom rejection, missing cluster metadata, and size rejection.

Exit criteria:

- A test can append an episode cluster and read it back.
- Malformed syntax does not corrupt the store.
- Existing OmegaClaw tests still pass or the prototype tests pass in isolation.

Milestone 3 (Step 2: add simple query functions). *Implement text-based query functions before attempting full AtomSpace indexing.*

Tasks:

1. Query by id: return all lines mentioning the id.
2. Query by cluster id: return the complete cluster.
3. Query by type: return clusters beginning with (`Episode ...`), (`Commitment ...`), (`OpenQuestion ...`), etc.
4. Query by About: return clusters containing (`About <id> <entity>`).

5. Query by status: *active, blocked, resolved, stale, distilled*.
6. Query by epistemic role: *observed-event, quoted-utterance, hypothesis, decision, action-boundary, derived-belief*.
7. Add bounded output limits for prompt safety.

Exit criteria:

- The agent can retrieve active commitments and active questions.
- Queries are deterministic and bounded.
- Query failure returns explicit error feedback, not an empty hallucination trap.

Milestone 4 (Step 3: prompt-view selection). *Generate a compact prompt-facing medium-memory view.*

Tasks:

1. Add `getMediumMemoryContext` in MeTTa or Python.
2. Select active commitments, active questions, relevant boundaries, and recent high-value episodes.
3. Insert the resulting bounded text into `getContext` after history or before history.
4. Add a config knob: `mediumMemoryMaxContextChars`.
5. Default the feature off until tests pass; then enable for ProtomegaTron only.

Exit criteria:

- A controlled mock run shows the prompt contains selected MM facts.
- Idle/no-input loops still do not trigger backend calls.
- Prompt size remains bounded and observable in logs.

Milestone 5 (Step 4: selective extraction protocol). *Teach ProtomegaTron when to write medium-memory atoms.*

Tasks:

1. Add prompt guidance: write MM atoms only for commitments, open questions, decisions, hypotheses, important observations, and boundaries.
2. Add prompt guidance: separate observed events and quoted claims from derived beliefs; do not write a conversational quote as a naked fact.
3. Add a skill such as `medium-remember` that appends a validated cluster.
4. Create examples for common cases: Ben request, completed artifact, blocker, design hypothesis, decision, stale belief correction.
5. Require explicit epistemic roles for quoted utterances, hypotheses, decisions, and derived beliefs.
6. Add tests using the mock LLM or deterministic helper outputs.

Exit criteria:

- A fresh Ben request creates an episode and commitment cluster.
- A design assertion creates a speech event and quoted claim; an optional derived belief is added only with truth value and evidence links.
- A completion update marks the commitment resolved and links an artifact.
- Routine greetings do not create medium-memory atoms unless they establish a durable fact or task.

Milestone 6 (Step 5: distillation). Add a non-destructive distillation pass.

Tasks:

1. Select old resolved commitments and stale questions as candidates.
2. Summarize candidates into a ChromaDB **remember** string or project Markdown entry, with provenance links.
3. Mark source atoms as distilled but do not delete them.
4. Add a report command showing what would be distilled before writing.

Exit criteria:

- Distillation can be run manually and reviewed.
- The ChromaDB memory or project record links back to source atom ids.
- No atom is deleted in version 1.

Milestone 7 (Step 6: richer PeTTa/MeTTa reasoning). After the file-backed store is stable, load selected clusters into a PeTTa/MeTTa reasoning context for consistency checks, simple inference, and later PLN inference over the memory AtomSpace.

Tasks:

1. Load active question and commitment atoms into a temporary AtomSpace.
2. Add rules for detecting contradictions, e.g. active and resolved status on the same id.
3. Add rules for surfacing overdue or blocked commitments.
4. Add rules linking episodes, artifacts, and formalization notes.
5. Add PLN-oriented representations for uncertain interpretive atoms such as hypotheses, importance estimates, confidence, and evidence relations.

Exit criteria:

- A test detects inconsistent statuses.
- A test retrieves all active commitments about a target project.
- A test runs at least one PLN-style inference over loaded medium-memory atoms.
- The reasoning layer remains optional; basic query still works without it.

9 Validation plan

9.1 Unit tests

Minimum tests:

1. Append valid atom cluster.
2. Reject malformed atom.
3. Reject oversized atom.
4. Query by id.
5. Query by type.
6. Query by about-entity.
7. Query by status.
8. Generate bounded prompt context.

9.2 Integration tests

Use mock channels and mock providers first:

1. Simulate a Ben request.
2. Confirm an episode and commitment cluster are added.
3. Simulate completion.
4. Confirm the commitment status changes to resolved and links an artifact.
5. Confirm prompt context includes the active item before completion and omits or compresses it after completion.

9.3 Live supervised smoke

Only after mock tests pass:

1. Run ProtomegaTron in supervised Telegram mode.
2. Send one explicit memory-worthy request.
3. Confirm the reply is concise and the medium-memory atom cluster is correct.
4. Confirm no repeated idle backend calls occur.
5. Stop or leave running only under the already-approved supervision policy.

10 Failure modes and mitigations

Risk 1 (Schema sprawl). *Too many predicates make memory less useful. Mitigation: keep version 1 to episode, entity, commitment, question, hypothesis, boundary, artifact, provenance, status, and about-links.*

Risk 2 (False formality). *Bad atoms can look authoritative. Mitigation: require provenance and confidence/status fields for interpretive atoms.*

Risk 3 (Prompt bloat). *A growing medium memory could overwhelm context. Mitigation: bounded prompt views, not full-store injection.*

Risk 4 (Write amplification). *If every turn writes atoms, the store becomes noisy. Mitigation: selective atomization guidance and tests that routine chatter writes nothing.*

Risk 5 (Stale commitments). *Resolved or obsolete atoms may keep resurfacing. Mitigation: explicit statuses and distillation markers.*

11 Recommended first engineering patch

The smallest useful patch is:

1. Add `src/medium_memory.py` with `append`, `tail`, and simple query.
2. Add `src/medium_memory.metta` exposing `medium-remember`, `medium-query`, and `medium-active`.
3. Add a config flag `enableMediumMemory` defaulting to `false`.
4. Add tests using a temporary medium-memory file.
5. Enable the feature only in the ProtomegaTron local wrapper after tests pass.

This patch creates value without requiring a new database, new credentials, a new service, or a global change in OmegaClaw behavior.

12 Open design questions

1. Should MM be upstreamed into OmegaClaw Core after the local prototype, or remain ProtomegaTron-specific until the ontology stabilizes?
2. Should the first query implementation remain text-based, or should it load the atom log into PeTTa on every query despite possible latency?
3. What is the right decay policy: age-based, status-based, attention-based, or explicit distillation only?
4. Should Hyperseed formalization notes be generated from MM atoms as a semi-automatic paper-writing substrate?